

第 26 章 NFS 客户端与远程文件系统

从 `server:/path` 到业务用户读写：把网络、export、协议、挂载、身份和权限放进同一条证据链。

对象模型操作语义验证诊断经典任务

大号阅读版

本章阅读导航 先抓住一条主线：NFS 不是“把远端目录写进一条 mount 命令”，而是让一个远端 export 经过协议和授权进入本机目录树。任何一步都需要独立证据，尤其不能把“服务器可达”“已经挂载”和“业务用户能写”视为同一状态。

01 识别远端源分清服务器、export、`server:/path` 和本地挂载点

02 选择协议证据根据 NFSv3/v4 选择 `showmount`、`rpcinfo` 或 2049 证据

03 建立当前挂载用 `mount -t nfs` 改变当前名称空间

04 读回实际状态用 `findmnt` 与 `nfsstat -m` 读取源、版本和选项

05 建立持久声明用 NFS fstab 条目和 `_netdev` 描述可重建状态

06 以目标身份验收比较 UID/GID、远端权限、`root_squash` 与真实读写

专题地图 - **知识专题** 从远端 export 到本地目录：先分清对象 - **知识专题** NFSv3 与 NFSv4：只掌握会改变调查路径的差异 - **知识专题** UID/GID、远端权限与 `root_squash` - **操作专题** 建立临时挂载、读取当前证据、写入持久声明 - **诊断专题** 挂载失败、身份权限异常与远端失联 - **经典任务** 持久挂载并验证业务身份；按层定位拒绝与权限问题

阅读时持续回答 1. 当前正在判断网络、export、协议、挂载还是权限？ 2. `showmount` 在这个版本场景中能证明什么？ 3. 当前内核实际挂载的源、版本和选项是什么？ 4. fstab 条目能否在当前挂载被移除后重新建立？ 5. 访问进程的数值 UID/GID 和附加组是什么？ 6. `root_squash` 是故障，还是预期的安全边界？ 7. 远端失联时，等待、报错和强制卸载各有什么风险？ 8. 现有证据能证明到哪一层，下一条最有区分度的证据是什么？

章节边界：本地挂载通用机制留给第 24 章；按需触发和空闲卸载留给第 27 章；本章不完整展开 NFS 服务端配置。

第 26 章 · 正文

一台服务器上的目录可以通过 NFS 暴露给网络中的客户端。客户端挂载以后，用户看到的仍是一棵普通目录树，但目录中的文件对象、权限判断和可用性已经跨越网络边界。正因为界面看起来与本地文件系统相似，NFS 故障也很容易被误判：服务器能解析不代表 export 允许当前客户端，目录已经挂载不代表业务用户能写，root 可以列目录不代表应用身份正确，当前挂载成功也不代表系统重新启动后仍会建立同一关系。

本章从客户端视角建立一条完整证据链：先确定服务器和 export，再确认协议版本与访问路径，建立当前挂载，写入持久声明，最后以目标用户的数值 UID/GID 和实际读写行为验收。服务端 `/etc/exports`、`exportfs`、服务启停和防火墙配置只作为诊断时需要向服务端管理员索取的证据，不在本章完整展开；按需挂载留给下一章《autofs 自动挂载》。

概念

NFS export 是服务端向特定客户端开放的远端目录及其访问策略。它不是客户端本地目录，也不等同于文件本身；客户端必须通过服务端提供的 NFS 名称空间引用它。判断 export 时既要核对路径，也要核对允许的客户端范围、只读或读写策略和安全方式。

概念

客户端源 `server:/path` 是客户端对远端 export 的引用。冒号左侧确定服务器，右侧是服务器向 NFS 客户端呈现的路径；它与本机是否存在同名路径无关。源格式正确只是对象正确，仍不能证明网络、协议或授权已经成立。

概念

NFSv3 与 NFSv4 的区别在客户端最直接地改变发现和端口证据。NFSv3 常通过 rpcbind 查找 MOUNT 与其他 RPC 服务；NFSv4 的主路径更集中，通常以 TCP 2049 和已知 NFSv4 路径为核心。版本不是装饰性选项，而是决定下一条证据的分流条件。

概念

身份映射 是把客户端访问进程的身份用于服务端权限判断的机制。常见 `sec=sys` 场景主要依赖数值 UID、主 GID 和附加组，而不是用户名的外观；两端同名用户若数值不同，仍可能被当作不同身份。

概念

`root_squash` 是服务端常见的安全默认：来自客户端 UID/GID 0 的请求会被映射成匿名身份，避免客户端 root 自动取得服务端 root 权限。客户端 root 不能写并不自动表示 NFS 故障，关键是题目要求的业务身份能否完成目标操作。

概念

远端文件系统状态 至少分为远端信息、当前挂载、持久声明和业务功能四层。`showmount`、`findmnt`、`fstab` 与目标用户读写分别回答不同问题；上一层成功不能自动证明下一层正确。

操作语义以下入口分别改变当前挂载、读取挂载证据、发现传统 export、读取 RPC 注册、描述持久状态和验证访问身份。先明确命令作用对象，再选择参数。

`mount -t nfs`

SYNOPSIS

```
mount -t nfs [-o option[,option...]] server:/path mountpoint
```

建立或重建当前 NFS 挂载关系，把远端源接入本地目录树。

重要参数 / 形式

server:/path

远端服务器与 export 路径的组合，冒号不可省略。

-t nfs

明确使用 NFS 文件系统助手。

-o vers=3 / vers=4.2

题目明确版本、兼容性已知或需要诊断版本分支时固定版本。

rw / ro

请求读写或只读挂载；显示为 **rw** 仍不证明业务身份有写权限。

findmnt

SYNOPSIS

```
findmnt [-T path] [-t type] [-o columns]
findmnt --verify --verbose
```

从当前挂载表或 fstab 读取源、目标、类型和选项，是本章最主要的结构化验证入口。

重要参数 / 形式

-T /srv/project

按路径定位其所属挂载，适合精确确认目标对象。

-o SOURCE,TARGET,FSTYPE,OPTIONS

只输出最能区分假设的字段。

--verify --verbose

静态检查 fstab 语法与可解析性；不等同于真实远端挂载成功。

showmount / rpcinfo

SYNOPSIS

```
showmount -e server
rpcinfo -p server
```

前者查询传统 MOUNT 协议提供的 export 列表，后者读取 rpcbind 注册的 RPC 程序，主要服务于 NFSv3 调查。

重要参数 / 形式

showmount -e

列出传统 MOUNT 服务愿意报告的 export；v4-only 服务器可能不提供该证据。

`rpcinfo -p`

查看 RPC 程序与端口注册；端口存在不证明 export 允许当前客户端。

版本边界

`showmount` 失败只能证明传统查询失败，不能单独否定 NFSv4。

`nfsstat -m`

SYNOPSIS

```
nfsstat -m
```

读取当前 NFS 挂载的实际版本、传输方式、安全方式和协商选项。

重要参数 / 形式

`-m`

显示已挂载 NFS 文件系统及其挂载参数。

`vers=` / `proto=` / `sec=`

确认实际协议版本、传输和认证方式。

`hard` / `soft`

读取远端失联时的重试语义；不能只为避免等待就默认改成 `soft`。

NFS fstab 条目

SYNOPSIS

```
server:/path mountpoint nfs defaults,_netdev 0 0
```

描述可由系统重新建立的持久 NFS 挂载，不是当前挂载本身。

重要参数 / 形式

`nfs`

文件系统类型。

`_netdev`

显式标记网络依赖；不保证 DNS、服务器或业务权限正确。

`defaults`

采用常规默认选项；仅在题目或业务语义明确时增加改变生命周期的选项。

`id` / `getent` / `ls -ln`

SYNOPSIS

```
id user
getent passwd user
getent group group
ls -ldn path
```

建立目标访问身份、NSS 解析结果和远端对象数值属主的证据。

重要参数 / 形式

`id user`

读取 UID、主 GID 和附加组。

`getent`

通过当前 NSS 配置解析用户和组，而不是只读取本地文件。

`ls -ln`

以数值 UID/GID 显示属主，避免用户名映射掩盖数值不一致。

`umount` / `fuser`

SYNOPSIS

```
fuser -vm mountpoint
umount mountpoint
```

调查占用并正常解除当前挂载，适合持久条目重建验证和受控维护。

重要参数 / 形式

`fuser -vm`

列出引用挂载点的进程、用户和访问类型。

`umount`

正常解除挂载；先让 Shell 和应用离开目标路径。

强制或 lazy 卸载

只属于已经评估未完成 I/O 和数据风险的应急分支，不是默认答案。

[知识专题] 从远端 export 到本地目录：先分清对象

面对一个 NFS 任务，最顺的切入方式不是先试几种 `mount` 参数，而是把服务端、远端源、当前挂载和本地路径分开。只有对象明确，后续报错才能定位到网络、协议、export、挂载或权限中的某一层。

① [知识点] Export、远端源和挂载点是三个不同对象

服务端 export 是服务器公布并授权的目录。客户端引用它时使用以下形式：

```
<server>:/<export-path>
```

例如：

```
storage.example.com:/exports/project
```

本地挂载点则是客户端目录，例如 `/srv/project`。三者关系是：

```
服务端 export /exports/project
      ↓ 客户端写成远端源
storage.example.com:/exports/project
      ↓ 挂载到本地目录
/srv/project
```

远端源中的路径必须按服务器提供的 NFS 名称空间填写。客户端本地是否也存在 `/exports/project` 与远端源无关。路径拼写错误、把本地路径误写进源字段，或漏掉主机名后的冒号，都会使任务从一开始就指向错误对象。

② [知识点] 挂载建立名称空间接入，不复制远端数据

挂载成功后，对 `/srv/project` 的文件操作被交给 NFS 客户端和远端服务器。卸载不会删除服务端数据，只是解除本地目录与远端文件系统的关联。

如果本地挂载点原来非空，挂载期间原内容会被远端文件系统遮蔽，卸载后才重新出现。这个通用机制主归属第 24 章《挂载、fstab、Swap 与启动持久性》；本章只强调考试和排错中的直接后果：挂载前先确认目标目录是否已有需要保留的内容，不要把“看不到”误判为“被删除”。

③ [知识点] 发现、当前挂载、持久声明和功能是四层状态

状态层	问题	典型证据	证据边界
远端信息	服务器和 export 是否合理	<code>getent hosts</code> 、 <code>showmount -e</code> 、服务端证据	不证明一定能挂载
当前状态	当前内核是否已有正确 NFS 挂载	<code>findmnt</code> 、 <code>nfsstat -m</code>	不证明重启后存在
持久状态	是否存在正确持久声明	<code>/etc/fstab</code> 、 <code>findmnt --verify</code> 、重建测试	不证明用户权限正确
功能状态	指定身份能否执行目标操作	<code>sudo -u</code> 读写测试、 <code>ls -ln</code>	只证明已测试的身份和操作

稳定的答案总是按层验收，而不是看到 `mount` 返回零就结束。

[Cheatsheet] `server:/path` 是远端源；本地目录只是接入点；`showmount` 看传统 export 信息，`findmnt` 看当前关系，`fstab` 看持久声明，目标用户读写才是功能终态。

[知识专题] NFSv3 与 NFSv4：只掌握会改变调查路径的差异

NFS 版本差异很多，但本章只保留会直接影响客户端发现、端口判断、路径和验证的部分。不要把协议版本当作背诵题；它真正决定的是下一条最有区分度的证据。

① [知识点] 默认可以协商，明确要求或排错时再固定 `vers=`

典型挂载可以不写版本：

```
mount -t nfs storage.example.com:/exports/project /srv/project
```

Linux NFS 客户端会与服务器协商可用版本。题目明确要求某版本、服务器兼容性已知，或需要验证某个版本分支时，才显式指定：

```
mount -t nfs -o vers=3 storage.example.com:/exports/project /srv/project
mount -t nfs -o vers=4.2 storage.example.com:/exports/project /srv/project
```

命令中写了 `vers=` 仍要在操作后读取实际挂载证据。未经验证，不把“命令没有报错”扩大为“实际使用了预期的所有参数”。

② [知识点] NFSv3 依赖更多 RPC 辅助服务，NFSv4 的主路径更集中

NFSv3 通常通过服务端 `rpcbind` 查找 MOUNT、NFS 和其他辅助 RPC 程序；除 NFS 服务外，还可能涉及 `mountd`、锁和状态相关服务。服务端的部分端口可能动态分配，因此只测试 TCP 2049 不能覆盖完整的 NFSv3 链路。

NFSv4 把更多功能整合进协议，常见主数据路径使用 TCP 2049，不需要客户端通过传统 MOUNT 协议获得同样的信息。由此形成诊断分流：

```
明确或疑似 NFSv3
→ rpcbind/MOUNT/NFS 辅助 RPC 证据

明确或疑似 NFSv4
→ 名称/路由/TCP 2049/已知 NFSv4 路径/实际挂载证据
```

不要为了“让 NFS 工作”在客户端盲目启用所有 RPC 服务。先确定使用的版本和失败层，再处理真正缺失的组件。

③ [知识点] `showmount` 有版本边界，空结果不是“服务器没有 NFS”的定论

`showmount -e <server>` 查询的是服务器 MOUNT 协议提供的 `export` 列表。它对 NFSv3 环境很有价值：

```
showmount -e storage.example.com
```


但仅提供 NFSv4 的服务器可能不运行 rpcbind/MOUNT 服务，`showmount` 因而可能超时、被拒绝或无法返回列表。此时正确判断是：

```
showmount 无结果
→ 传统 MOUNT 查询不可用
≠ 已证明服务器没有 NFSv4 export
```

如果题目已经给出准确的 NFSv4 源，应继续检查名称解析、TCP 2049、路径语义和实际挂载报错。若需要从 NFSv4 服务器浏览导出树，是否能挂载 `server:/` 取决于服务端如何设置 NFSv4 根和可浏览路径，不能作为所有服务器都支持的固定技巧。

[Cheatsheet] 默认先协商，必要时用 `vers=` 固定；v3 看 rpcbind/MOUNT 及辅助 RPC，v4 主路径更集中；`showmount` 失败只证明传统查询失败，不能否定 v4。

[知识专题] UID/GID、远端权限与 `root_squash`

NFS 最常见的“已经挂载但不能用”并不是挂载问题，而是身份和权限问题。最有效的调查方法不是反复 `chmod` 本地挂载点，而是把访问进程的数值身份、远端文件的数值属主和服务端 `export` 策略放在同一条链上。

① [知识点] `sec=sys` 常按客户端数值 UID/GID 传递身份

对常见 AUTH_SYS（挂载选项常显示为 `sec=sys`）访问，客户端请求携带进程的 UID、主 GID 和附加组。服务端用这些数值与服务端文件系统上的 `owner`、`group` 和权限位进行判断。

客户端侧先建立访问身份证据：

```
id project1
getent passwd project1
getent group project
```

文件显示可以同时观察名称和数值：

```
ls -l /srv/project
ls -ln /srv/project
stat -c 'uid=%u gid=%g mode=%A name=%n' /srv/project/path
```

用户名相同但 UID 不同，会被服务端视为不同身份；用户名不同但 UID 相同，数值权限判断可能仍把它们视为同一身份。RHCSA 任务中优先用数值证据解释权限，不依赖“看起来名字一样”。

② [知识点] 挂载成功不会绕过服务端文件权限

挂载建立了访问路径，但不会自动授予写权限。最终结果可能同时受以下因素影响：

```
export 是否只读或可写
→ 服务端文件 owner/group/mode/ACL
→ 客户端进程 UID/GID/附加组
→ 安全方式 sec=
→ root 或其他身份映射
```

本地挂载点在挂载前的 owner 和 mode，不会替代远端根目录的权限。它们主要影响挂载前的本地目录，以及客户端是否能沿父目录路径到达挂载点。挂载后看到的根目录属主和权限来自远端文件系统。

③ [知识点] `root_squash` 让客户端 root 不自动拥有服务端 root 权限

服务端 export 的常见默认策略 `root_squash` 会把来自客户端 UID/GID 0 的请求映射为匿名身份。这样做是为了防止任意客户端 root 直接成为共享存储上的服务端 root。

典型表现包括：

- root 能列目录但不能在受限目录中创建文件；
- root 创建的文件在服务端显示为匿名 UID/GID；
- 业务用户能够写，但 root 的测试结果不同；
- `chown` 等特权操作被拒绝。

不要把 `no_root_squash` 当作“写入失败”的默认修复。正确做法是明确题目要求由哪个业务身份访问，再校对两端 UID/GID、附加组、export 读写模式和远端文件权限。匿名身份的具体用户名可能因系统配置而异，应以数值 UID/GID 和服务端配置为准。

[Cheatsheet] 先 `id`，再 `ls -ln`；挂载成功不增加权限；客户端 root 受 `root_squash` 约束是安全默认，不用 `no_root_squash` 掩盖身份设计错误。

[操作专题] 准备客户端并建立临时 NFS 挂载

临时挂载适合先验证远端源和协议，也适合在写入持久配置前建立基线。操作顺序应从“已有状态”开始：确认目标目录是否已经挂载、确认软件能力和名称解析，然后才创建目录并挂载。

① [操作] 确认 `nfs-utils` 和帮助入口

作用对象：客户端 NFS 辅助程序和诊断工具。基本语义：RHEL 9 使用 `nfs-utils` 提供 `mount.nfs`、`showmount`、`nfsstat` 等能力。典型操作：

```
rpm -q nfs-utils
dnf install nfs-utils
```

安装后可使用本地手册确认语义：

```
man 5 nfs
man 8 mount.nfs
man 8 showmount
man 8 nfsstat
```

边界：安装包成功不证明服务器可达；通常也不需要为了 NFSv4 挂载手工启动一个“客户端 NFS 服务”。具体依赖由挂载流程和 systemd 按需处理。

② [操作] 调查名称、已知 export 和已有挂载

作用对象：服务器名称、目标路径和客户端当前名称空间。基本形式：

```
getent hosts storage.example.com
findmnt -T /srv/project
showmount -e storage.example.com
```

在执行 `showmount` 前先明确其版本边界。若服务器是 v4-only，`showmount` 失败不是停止调查的理由。若任务使用 NFSv3，可补充：

```
rpcinfo -p storage.example.com
```

`rpcinfo` 输出用于确认服务端 rpcbind 注册的 RPC 程序；它不是权限测试，也不证明 export 允许当前客户端。

③ [操作] 创建挂载点并执行 `mount -t nfs`

作用对象：当前内核挂载关系。基本形式：

```
mkdir -p /srv/project
mount -t nfs storage.example.com:/exports/project /srv/project
```

显式版本示例：

```
mount -t nfs -o vers=4.2 \
storage.example.com:/exports/project /srv/project
```

关键检查：

```
findmnt -T /srv/project -o SOURCE,TARGET,FSTYPE,OPTIONS
nfsstat -m
```

边界：

- 如果目标已经挂载，先读取现状，不重复叠加挂载；

- 不用 `-o soft` 作为“防止卡住”的默认选项；
- 不把 `rw` 显示为挂载选项扩大解释为业务用户一定能写；
- 临时挂载在重启后不会自动恢复。

[Cheatsheet] 包：`nfs-utils`；源：`server:/path`；当前挂载：`mount -t nfs`；操作后用 `findmnt` 和 `nfsstat -m` 读回实际状态。

[操作专题] 从 `findmnt`、`mount` 和 `nfsstat` 读取当前证据

同一个挂载可以从多个工具观察，但工具回答的问题不同。优先选择能够直接区分假设的字段，而不是在冗长输出中寻找看起来熟悉的一行。

① [操作] 用 `findmnt` 建立源、目标、类型和选项映射

作用对象：当前挂载表与配置文件。典型形式：

```
findmnt -T /srv/project
findmnt -T /srv/project -o SOURCE,TARGET,FSTYPE,OPTIONS
findmnt -t nfs,nfs4
```

`-T` 按任意路径查找其所属挂载，即使给的是挂载点下的文件路径也能定位。排错时重点读：

- `SOURCE` 是否是预期服务器和路径；
- `TARGET` 是否是预期本地目录；
- `FSTYPE` 是否是 NFS；
- `OPTIONS` 中是否包含预期的版本、安全方式和读写模式。

`findmnt` 无结果时，目录仍可能存在，只是它不属于单独的当前挂载。

② [操作] 把 `mount` 作为兼容的快速视图

以下命令可以浏览当前 NFS 挂载：

```
mount -t nfs,nfs4
```

它适合人工快速查看，但字段筛选和按路径定位通常不如 `findmnt` 清晰。不要使用不精确的 `mount | grep nfs` 结果作为唯一证据：服务器名、路径或目录中可能恰好包含匹配字符串。

③ [操作] 用 `nfsstat -m` 查看 NFS 专属挂载信息

```
nfsstat -m
```

该视图可帮助确认：

- 实际 NFS major/minor 版本；
- 传输协议；
- `sec=` 安全方式；
- `hard / soft`、超时和重传相关选项；
- 读写、缓存和其他协商参数。

`nfsstat -m` 只能证明当前 NFS 客户端挂载参数，不能证明服务端数据内容正确、应用没有锁冲突或业务用户具备权限。验证应继续进入目标身份和真实文件操作。

[Cheatsheet] 精确路径用 `findmnt -T`；快速浏览可用 `mount -t nfs,nfs4`；协议和 NFS 专属选项用 `nfsstat -m`；三者都不是业务功能测试。

[操作专题] 用 `/etc/fstab` 建立持久 NFS 挂载

持久挂载不是“把成功命令抄进文件”这么简单。它需要声明可解析的源和挂载点，明确网络依赖，并通过卸载后重建来证明条目确实能够独立建立当前状态。

① [操作] 编写 NFS fstab 条目并解释 `_netdev`

示例条目：

```
storage.example.com:/exports/project /srv/project nfs defaults,_netdev 0 0
```

六个字段分别是：远端源、本地挂载点、文件系统类型、挂载选项、dump 字段和 fsck 顺序。此处只说明 NFS 特有部分；通用六字段主归属第 24 章。

`_netdev` 明确该文件系统依赖网络，使 `systemd` 按远程文件系统处理相关顺序。它不表示：

- DNS 一定正确；
- 网络在线目标一定代表服务器可访问；
- 服务器宕机时系统绝不会等待；
- 挂载会在失败后按业务期望自动恢复；
- 用户权限正确。

除非题目明确要求，不随意添加 `nofail`、`bg`、`soft` 或 `x-systemd.automount`。这些选项会改变启动、错误返回或生命周期语义；按需挂载的完整设计留给第 27 章。

② [操作] 先静态核对，再由 `fstab` 重建当前状态

变更前保留基线：

```
cp -a /etc/fstab /etc/fstab.before-rhcsa26
```

编辑后执行：

```
findmnt --verify --verbose
systemctl daemon-reload
```

静态核对通过后，使用条目建立挂载：

```
mount /srv/project
findmnt -T /srv/project -o SOURCE,TARGET,FSTYPE,OPTIONS
```

如果之前已经手工挂载，为证明 `fstab` 自身可工作，应在确认没有进程占用后卸载，再重建：

```
umount /srv/project
mount -a
findmnt -T /srv/project -o SOURCE,TARGET,FSTYPE,OPTIONS
```

`mount -a` 可能处理系统中的其他 `fstab` 条目。在真实生产系统中应先评估全局影响；考试实验环境中仍要先运行 `findmnt --verify`，避免一个错误条目影响整体测试。

③ [验证] 当前、持久和功能分层验收

建议验收矩阵：

```
# 当前关系
findmnt -T /srv/project -o SOURCE,TARGET,FSTYPE,OPTIONS

# NFS 协议和选项
nfsstat -m

# 指定身份
id project1

# 最小功能测试
sudo -u project1 sh -c '
    umask 022
    printf "%s\n" "nfs-client-check" > /srv/project/.client-check
    cat /srv/project/.client-check
    printf "%s\n" "append" >> /srv/project/.client-check
    rm -f /srv/project/.client-check
'
```

不要编造输出，也不要把 `root` 测试替代业务用户测试。若题目只要求只读访问，则使用 `test -r`、`cat` 或读取指定文件，不额外创建文件改变远端数据。

[Cheatsheet] `fstab`： `server:/path target nfs defaults,_netdev 0 0`；先 `findmnt --verify`，再 `daemon-reload`；卸载后由 `mount -a` 重建；最后按目标身份做真实功能测试。

[诊断专题] 服务器可达，但 NFS 挂载失败

“服务器可达”只排除了很小一部分问题。诊断必须先记录原始报错，再选择能最大程度区分假设的下一条证据。不要因为 `ping` 成功就跳过协议和 `export`，也不要因为 `showmount` 失败就直接修改路径。

① [诊断] 名称、路由、端口和协议层分开取证

症状： `mount` 报名称解析失败、无路由、连接拒绝或超时。当前证据：原始命令和完整错误文本。下一条证据：

```
getent hosts storage.example.com
ip route get <resolved-address>
```

根据版本继续：

NFSv4 路径
→ TCP 2049 和服务器 NFS 服务证据

NFSv3 路径
→ 服务端 `rpcbind`、`MOUNT`、NFS 及辅助 `RPC` 注册/防火墙证据

客户端可以使用允许的连接检查工具，但不要将“端口通”扩大为“`export` 允许”。连接被拒绝与超时也不同：前者说明目标主动拒绝或无监听，后者更可能涉及过滤、路由、服务无响应或丢包。

② [诊断] 按 v3/v4 边界解释 `showmount` 和 `rpcinfo`

症状： `showmount -e` 没有返回预期列表。假设 A：服务器或网络确实不可用。假设 B：服务器是 v4-only，不提供传统 `MOUNT` 查询。假设 C：`rpcbind`/`MOUNT` 被防火墙过滤。最有区分度的证据：题目指定版本、服务端版本配置、TCP 2049、`rpcinfo -p` (v3) 和已知源的实际挂载报错。

不要为了让 `showmount` 成功去改变 v4-only 服务器设计。`showmount` 是发现工具，不是 NFSv4 服务健康的唯一验收。

③ [诊断] `access denied by server` 指向 `export` 或安全策略层

典型报错中的“`server denied`”意味着请求已经到达服务端，服务端拒绝了挂载或访问。调查顺序：

```
客户端请求的 server:/path 是否准确
→ 该路径在目标 NFS 版本的名称空间中是否有效
→ export 是否允许当前客户端地址/网段
→ export 是否要求不同 sec= 安全方式
→ 服务端是否已加载最新 export 配置
```

客户端对本地挂载点执行 `chmod 777` 不能修改服务端 `export` 许可。最小修复应发生在错误的那一层：纠正客户端源路径，或由服务端管理员修正 `export` 范围和安全策略，然后重新挂载并分层验证。

[Cheatsheet] 先留原始报错；解析→路由→版本对应端口/RPC→路径→export 客户端范围→`sec=`；`access denied by server` 不是本地目录 mode 问题。

[诊断专题] 已经挂载，但属主、读写或 root 行为不符合预期

此类故障要保持同一测试身份。用 root 复现、再用普通用户修复，或先后切换多个用户，会混淆 UID/GID、附加组和 `root_squash` 三个变量。

① [诊断] 先用目标用户重现，并记录数值身份

症状：`Permission denied`、只能读不能写、创建文件属主异常。当前证据：指定用户执行的最小操作及错误。下一条证据：

```
id project1
sudo -u project1 test -r /srv/project && echo readable
sudo -u project1 test -w /srv/project && echo writable
ls -ldn /srv/project
```

`test -w` 是权限预判，不等同于创建一定成功；最终仍需按题目执行最小真实操作。

② [诊断] 比较数值 owner/group 和附加组

若目录由某组写入，除了主 GID，还要查看用户的附加组：

```
id project1
getent group project
ls -ldn /srv/project
```

需要服务端证据时，应让服务端管理员用数值形式检查远端目录和文件。若两端用户同名但 UID 不同，修复应是统一身份来源或调整正确的文件属主/组，而不是扩大权限位。

③ [诊断] 区分普通权限问题与 `root_squash`

如果只有客户端 root 行为异常，或 root 创建内容显示匿名属主，应检查 export 的 root 映射。判断链：

```
业务用户能否完成要求
→ root 是否本来就不是业务访问身份
→ 服务端是否使用 root_squash
→ 匿名 UID/GID 是否具备目标权限
```

题目要求普通用户读写时，root 被 squash 不构成故障。只有业务需求明确要求某种特权操作时，才需要重新设计服务端授权；也不应默认启用 `no_root_squash`。

[Cheatsheet] 固定测试身份；`id` 看 UID/GID/附加组；`ls -ln` 看数值属主；业务用户失败查权限链，只有 root 异常再查 `root_squash`。

[诊断专题] 远端不可用、请求等待与安全卸载边界

网络文件系统把本地文件操作变成远端请求。服务器失联时，应用可能不是立即报错，而是等待服务器恢复。排错人员若不了解这一点，容易反复终止进程、强制卸载或把 `soft` 写进所有配置，进而增加数据风险。

① [知识点] 默认 `hard` 语义更偏向等待远端恢复

常见 NFS 挂载使用 `hard` 语义：请求超时后继续重试，直至服务器响应。对正在访问挂载点的进程，这可能表现为系统调用长时间不返回，甚至处于不可中断 I/O 等待。

此时：

- `kill -9` 不一定能立即终止处于不可中断等待的任务；
- 新的 `ls`、`df`、`stat` 或 Shell 自动补全也可能触发远端访问并一起等待；
- 应优先恢复名称解析、网络路径或 NFS 服务，而不是制造更多访问请求。

② [安全边界] `soft` 不是通用的“避免卡死”方案

`soft` 类挂载在重试耗尽后向应用返回错误。对某些只读或可容忍失败的工作负载，经过应用和数据设计评估后可能使用；但对一般读写文件系统，它可能使应用收到部分失败、I/O 错误或产生数据完整性问题。

因此本章默认：

不因担心等待而自动添加 `soft`
不把调小 `timeo/retrans` 当作普适修复
先依据业务语义、应用容错和数据重要性评估

考试未明确要求时，保留安全默认并解释远端不可用行为。

③ [诊断] 先恢复远端链路，再处理占用和卸载

症状：挂载点命令等待，或 `umount` 报 `busy`。当前证据：当前 NFS 源、网络和占用情况。调查：

```
findmnt -T /srv/project -o SOURCE,TARGET,FSTYPE,OPTIONS
fuser -vm /srv/project
```

先让 Shell 离开挂载目录，正常停止正在使用该路径的应用，再尝试普通卸载：

```
cd /
umount /srv/project
```

`umount -f` 或 `lazy unmount` 会改变失败处理方式，可能隐藏仍在运行的引用或未完成 I/O，只能在已经评估数据风险、正常恢复不可行的应急场景使用，不能作为考试和日常操作的首选答案。

恢复后重新验证：

- 网络和服务器恢复
- 当前挂载是否恢复或需要重建
 - 文件系统读写是否正常
 - 应用是否有未完成事务或损坏
 - 持久配置是否仍符合目标

[Cheatsheet] hard 失联可能长期等待；D 状态不能靠反复 `kill -9`；`soft` 有数据风险；先恢复远端和正常停止使用者，再考虑卸载，强制手段只作受控应急分支。

[操作专题] 把 NFS 调查写成可重复的证据清单

考试中最容易丢分的不是不知道 `mount`，而是修改过多、验证不足。下面的清单把每一步限制为一个问题，既便于手工操作，也为后续 Ansible 自动化准备稳定的状态模型。

① [操作] 变更前建立基线

```
rpm -q nfs-utils
getent hosts storage.example.com
findmnt -T /srv/project
```

记录：目标目录是否存在、是否已挂载、当前源和选项、是否已有 `fstab` 条目。不要先删除旧条目或覆盖目录。

② [操作] 变更只改变必要对象

缺包 → 安装 `nfs-utils`
缺挂载点 → `mkdir -p`
源或选项错 → 纠正 `mount/fstab`
身份错 → 统一 `UID/GID` 或正确属主/组
`export` 拒绝 → 服务端修正授权

避免把一个层的故障用另一层的高风险配置绕过。例如：`export` 拒绝不靠 `chmod 777`，`root squash` 不靠 `no_root_squash`，远端超时不靠默认 `soft`。

③ [验证] 建立证据矩阵而不是一句“成功”

验收对象	推荐证据
软件能力	<code>rpm -q nfs-utils</code>
名称解析	<code>getent hosts</code>
当前源和目标	<code>findmnt -T</code>
协议和选项	<code>nfsstat -m</code>

验收对象	推荐证据
持久配置	<code>findmnt --verify</code> 、卸载后重建
访问身份	<code>id</code> 、 <code>getent</code>
数据功能	目标用户按题意读写
远端故障边界	日志、网络、应用等待和恢复复验

[Cheatsheet] 基线→最小变更→读回状态→重建持久状态→目标身份功能；任何工具只证明自己的那一层。

[经典任务] 配置持久 NFS 客户端挂载并验证读写身份

任务环境

你管理一台 RHEL 9 客户端。存储服务器提供以下已知远端源：

```
storage.example.com:/exports/project
```

要求在客户端将其挂载到：

```
/srv/project
```

业务用户 `project1` 已经存在。服务端管理员确认该 `export` 允许本客户端访问，并且预期 `project1` 具有读写权限。本题不允许修改服务端、不使用 `autofs`，也不把 `soft`、`nofail`、`no_root_squash` 或 `chmod 777` 作为捷径。

当前状态

- 不确定客户端是否已安装 NFS 工具；
- `/srv/project` 可能不存在；
- 当前没有已确认的正确挂载；
- `/etc/fstab` 中可能存在其他系统条目，不能破坏；
- 本会话没有真实服务器输出，所有命令都需要在实验环境执行后按实际结果判断。

目标终态

1. 客户端具备 NFS 挂载工具；
2. 当前挂载源、目标和类型准确；
3. `/etc/fstab` 中存在带 `_netdev` 的持久条目；
4. 卸载当前挂载后，通过持久配置可以重新建立；
5. 可以读取实际 NFS 版本和挂载选项；

- 6. `project1` 可以创建、读取、追加并删除测试文件；
- 7. 验证过程不把 `root` 的成功替代为业务用户成功。

验收证据

验收项	必须提供的证据
软件	<code>rpm -q nfs-utils</code>
解析	<code>getent hosts storage.example.com</code>
当前挂载	<code>findmnt -T /srv/project</code>
NFS 参数	<code>nfsstat -m</code>
fstab 静态核对	<code>findmnt --verify --verbose</code>
持久重建	卸载后 <code>mount -a</code> ，再查 <code>findmnt</code>
身份	<code>id project1</code>
功能	<code>sudo -u project1</code> 的最小读写测试

[参考解答] 按“调查—当前—持久—身份—功能”完成任务

① [调查] 保留已有状态并确认客户端能力

```
rpm -q nfs-utils || dnf install -y nfs-utils
getent hosts storage.example.com
findmnt -T /srv/project
```

如果 `findmnt` 已显示一个挂载，先比较其 `SOURCE`、`TARGET`、`FSTYPE` 和 `OPTIONS`，不要直接叠加。检查现有配置：

```
grep -nE '(^|[:space:])/srv/project([[:space:]]|$)' /etc/fstab
```

此处的 `grep` 只用于定位相关行，最终仍要解析和重建验证。

② [操作] 建立受控的当前挂载

确认挂载点不存在或可安全使用：

```
mkdir -p /srv/project
```

建立临时挂载：

```
mount -t nfs \
storage.example.com:/exports/project \
/srv/project
```

读回实际状态：

```
findmnt -T /srv/project -o SOURCE,TARGET,FSTYPE,OPTIONS
nfsstat -m
```

若挂载失败，保留原始错误，按名称/路由、版本、export 和安全方式分层调查；不要随机追加多个选项。

③ [配置] 写入并核对持久声明

先备份：

```
cp -a /etc/fstab /etc/fstab.before-rhcsa26
```

确保最终只保留一个目标条目：

```
storage.example.com:/exports/project /srv/project nfs defaults,_netdev 0 0
```

静态核对并让 systemd 重新运行生成器：

```
findmnt --verify --verbose
systemctl daemon-reload
```

如果静态检查指出其他既有条目问题，不应无授权修改与本题无关的系统配置；记录问题并只纠正本题引入的错误。

④ [验证] 证明 fstab 能独立重建挂载

先确认没有进程在使用测试目录：

```
fuser -vm /srv/project
```

在安全条件下：

```
cd /
umount /srv/project
mount -a
findmnt -T /srv/project -o SOURCE,TARGET,FSTYPE,OPTIONS
nfsstat -m
```

若不希望 `mount -a` 影响其他条目，也可以先使用 `mount /srv/project` 直接按目标匹配 `fstab`；但完整考试验收仍应保证 `fstab` 整体没有由本题引入的语法错误。

⑤ [验证] 使用业务身份完成真实读写

```
id project1
sudo -u project1 sh -c '
    set -eu
    file=/srv/project/.rhcsa26-client-check
    printf "%s\n" "created-by-project1" > "$file"
    cat "$file"
    printf "%s\n" "append-ok" >> "$file"
    tail -n 1 "$file"
    rm -f "$file"
'
```

如果失败：

```
id project1
ls -ldn /srv/project
findmnt -T /srv/project -o SOURCE,TARGET,FSTYPE,OPTIONS
```

然后请服务端管理员按数值 UID/GID 检查远端目录、`export` 读写模式和 `root/` 匿名映射。不要在客户端对挂载点执行 `chmod 777` 试图改变远端 `export` 策略。

⑥ [验收] 汇总而不扩大结论

最终可以声明的证据是：

- 软件包存在；
- 名称按 NSS 解析；
- 当前源、目标和类型符合任务；
- `nfsstat -m` 记录实际协议和选项；
- `fstab` 通过静态核对并能重建挂载；
- `project1` 完成了指定的创建、读取、追加和删除。

本章没有连接 live VM，因此不能声称这些命令已在真实 RHEL 9 环境跑通。

[经典任务] 诊断服务器可达但挂载被拒绝或写入权限不符

场景

客户端要使用：

```
files.example.com:/departments/ops
```

挂载点为 `/mnt/ops`。用户报告“服务器能通，但 NFS 不能用”。当前可能出现以下任一症状：

- `showmount -e` 超时；
- `mount` 报 `access denied by server`；
- 挂载成功，但用户 `opsuser` 创建文件时报 `Permission denied`；
- `root` 与 `opsuser` 的测试结果不同。

限制

- 不关闭客户端或服务端防火墙；
- 不关闭 SELinux；
- 不默认使用 `no_root_squash`；
- 不执行 `chmod 777`；
- 不反复尝试所有 NFS 版本；
- 必须说明每条证据正在区分哪两个假设。

验收

提交一条从症状到最小修复的调查链，并在修复后分别证明当前挂载、持久状态（如题目要求）和 `opsuser` 功能。

[参考解答] 先确定失败层，再做最小修复

① [调查] 固定原始命令、症状和目标版本

```
getent hosts files.example.com
findmnt -T /mnt/ops
```

记录题目是否明确版本。若未明确，先使用默认协商，不把随机固定版本作为第一步。

② [分流] `showmount` 超时

`showmount` 超时至少存在三种假设：服务器/网络不可用、传统 MOUNT/RPC 被过滤、服务器是 v4-only。
下一条证据应围绕实际版本：

若要求 NFSv3

→ `rpcinfo -p files.example.com`

→ 服务端 `rpcbind/MOUNT/NFS` 和防火墙证据

若要求或已知 NFSv4

→ TCP 2049、已知路径和 `mount` 原始错误

→ 不要求 `showmount` 必须成功

最小修复是恢复对应协议链，不是为了 `showmount` 改变服务器版本设计。

③ [分流] `access denied by server`

核对客户端请求：

```
mount -t nfs files.example.com:/departments/ops /mnt/ops
```

向服务端管理员索取：

- 目标路径在该 NFS 版本名称空间中的准确形式；
- `export` 允许的客户端主机或网段；
- `export` 的 `ro / rw` 和 `sec=`；
- 最新 `export` 配置是否已经加载。

最小修复只更正错误源路径或服务端授权。客户端挂载点 `mode` 不能解决服务端挂载拒绝。

④ [分流] 已挂载但 `opsuser` 不能写

```
findmnt -T /mnt/ops -o SOURCE,TARGET,FSTYPE,OPTIONS
nfsstat -m
id opsuser
ls -ldn /mnt/ops
sudo -u opsuser sh -c 'touch /mnt/ops/.ops-check'
```

比较：客户端 `opsuser` UID/GID/附加组、服务端远端目录数值属主、`export` 是否只读。如果 `root` 与 `opsuser` 不同，再检查 `root_squash`，但题目要求 `opsuser` 写入时应优先修正 `opsuser` 的身份和远端权限。

⑤ [修复后验证]

同一条源重新挂载

→ `findmnt` 确认当前源和目标

→ `nfsstat -m` 确认版本/选项

→ `opsuser` 执行题目要求的读写

→ 如要求持久化，再做 `fstab` 静态核对与重建

调查结论必须精确到层，例如“服务端 export 未允许本客户端地址”，而不是笼统写“网络问题”或“NFS 权限问题”。

[本章收束] 把“远端目录能用”还原成一条可验证链

NFS 客户端任务的核心不是记住一行 `mount`，而是始终知道当前证据属于哪一层。远端 export 通过 `server:/path` 被引用；协议版本决定发现和端口证据；当前挂载由 `findmnt` 与 `nfsstat -m` 读回；持久状态由 `fstab` 静态核对和卸载后重建证明；最终授权则由数值 UID/GID、远端权限、export 策略和目标用户实际操作共同决定。

可执行的工作方法

- 先保留原始症状和当前状态
- 明确 `server:/path` 与本地挂载点
 - 按 NFS 版本选择网络、端口和 RPC 证据
 - 建立或修正当前挂载
 - 读回实际源、版本和选项
 - 由 `fstab` 独立重建持久状态
 - 固定目标用户，比较 UID/GID 与远端数值权限
 - 做最小功能测试
 - 只修复已被证据定位的那一层

章末检查清单

- ☐ 我能说清 export、客户端源和本地挂载点的区别。
- ☐ 我不会把 `showmount` 当作所有 NFSv4 场景的唯一证据。
- ☐ 我能用 `findmnt` 和 `nfsstat -m` 读取当前挂载的实际状态。
- ☐ 我能解释 `_netdev` 能做什么，以及不能保证什么。
- ☐ 我会在修改权限前先固定测试身份并查看数值 UID/GID。
- ☐ 我知道 `root_squash` 往往是预期安全边界，而不是必须取消的故障。
- ☐ 我不会把 `soft`、`chmod 777`、`no_root_squash` 或强制卸载作为无调查默认答案。
- ☐ 我能分别验收当前状态、持久状态和业务功能。

主要判断表

现象或证据	它能支持的判断	它不能单独证明什么	下一条更有区分度的证据
<code>getent hosts</code> 返回地址	NSS 能解析服务器名称	NFS 端口、export 或权限正确	路由与版本对应的端口/RPC
<code>showmount -e</code> 返回 export	传统 MOUNT 查询成功	v4 实际挂载与业务权限正确	已知源的挂载与 <code>nfsstat -m</code>
<code>showmount</code> 超时或失败	传统 MOUNT 查询不可用	服务器没有 NFSv4	版本信息、TCP 2049、实际挂载报错

现象或证据	它能支持的判断	它不能单独证明什么	下一条更有区分度的证据
<code>findmnt</code> 显示 NFS	当前内核存在挂载关系	<code>fstab</code> 能重建、用户能读写	<code>nfsstat -m</code> 、 <code>fstab</code> 重建、用户测试
<code>fstab</code> 有正确条目	存在持久声明	当前已挂载、远端可达	<code>findmnt --verify</code> 后卸载重建
<code>rw</code> 出现在挂载选项	客户端请求了读写语义	<code>export</code> 与文件权限允许该用户写	固定业务用户的最小创建测试
<code>root</code> 不能写	<code>root</code> 请求未获得目标权限	业务用户也失败	业务用户测试与 <code>root_squash</code> 证据
访问挂载点长期等待	远端请求可能在重试	一定是 CPU 或本地磁盘故障	当前 NFS 源、网络、服务器与进程状态

向下一章交接

本章处理的是“声明后稳定存在”的 NFS 客户端挂载。需要在访问路径时才触发挂载、空闲后自动卸载，或用通配符映射多个远端目录时，进入第 27 章《autofs 自动挂载》。两章共享远端源和身份模型，但生命周期、配置对象和验证入口不同，不能把 `x-systemd.automount` 或 `autofs map` 机械混入本章答案。